

Apache Groovy

ОСНОВЫ ЯЗЫКА

Что такое Groovy

- Groovy работает на виртуальной машине Java
- Язык использует классы и библиотеки Java
- Программы можно писать как скрипты и приложения

```
println "Hello, Groovy!"
```

Вывод программы

```
Hello, Groovy!
```

•Groovy-скрипт

•Виртуальная машина
JVM

•Библиотеки Java

История Groovy

2003

- Начало разработки языка

• Начало проекта

2007

- Выпуск Groovy 1.0

• Стабильная версия

2015

- Проект верхнего уровня Apache

• Развитие в Apache

- Язык развивается как открытый проект для платформы JVM
- Современный Groovy сохраняет доступ к экосистеме Java

Java и Groovy

Java

```
public class Hello {  
    public static void main(String[] args) {  
        String name = "Alice";  
        System.out.println("Hello, " + name + "!");  
    }  
}
```

- Groovy подставляет name через интерполяцию \$name
- Java 25 тоже поддерживает компактные исходные файлы

Groovy

```
def name = "Alice"  
println "Hello, $name!"
```

Общий результат

Hello, Alice!

Отличительные особенности Groovy

- Лаконичный синтаксис и запуск программ как скриптов
- Динамическая типизация и средства статической проверки типов
- Интерполяция строк для подстановки значений
- Closures для передачи исполняемых блоков кода
- Удобное создание и обработка коллекций
- Интеграция с классами и библиотеками Java
- Перегрузка операторов для собственных типов
- Метaprogramмирование для изменения поведения программы
- Создание DSL – предметно-ориентированных языков

• Экосистема Java

• Динамические возможности

Запуск Groovy

- Локально выполняем файл командой groovy
- В groovysh проверяем выражения интерактивно
- Docker запускает Groovy в отдельной среде

```
// PowerShell: локальный запуск
groovy --version
groovy .\01\src\01_hello.groovy

// Интерактивная оболочка
groovysh
println "Hello, Groovy!"
2 + 3

// PowerShell: Docker
docker run --rm groovy:5-jdk21 groovy -e 'println "Hello, Groovy!'"
```

- В Docker-примере используется Groovy 5 с JDK 21

Переменные

def

- Динамическая типизация

- Тип определяется по значению
- `getClass()` показывает его во время выполнения

```
def name = "Alice"
def age = 20
def height = 1.75

println "Name: $name"
println "Age: $age"
```

```
println "Height: $height"

println "\nTypes:"
println name.getClass()
println age.getClass()
println height.getClass()
```

Фрагмент вывода

```
Name: Alice
Age: 20
Height: 1.75
```

Типы данных

BigDecimal

- Десятичные числа

- 3.14 по умолчанию – десятичное число BigDecimal
- Для double тип задают явно: например, 3.14d
- List, Map и Range создаются короткими литералами

```
def integer = 42
def decimal = 3.14
def flag = true
def text = "Groovy"
def list = [1, 2, 3]
def map = [name: "Alice", age: 20]
def range = 1..10
```

```
println "$integer -> ${integer.getClass()}"
println "$decimal -> ${decimal.getClass()}"
println "$flag -> ${flag.getClass()}"
println "$text -> ${text.getClass()}"
println "$list -> ${list.getClass()}"
println "$map -> ${map.getClass()}"
println "$range -> ${range.getClass()}"
```

Фрагмент вывода

```
42 -> class java.lang.Integer
3.14 -> class java.math.BigDecimal
```

Операции и преобразования

/ и intdiv

- Два способа деления

- 10 / 3 даёт дробный результат; 10.intdiv(3) – 3
- as Integer превращает строку "42" в число

```
def a = 10
def b = 3

println "Arithmetic operations:"
println "a + b = ${a + b}"
println "a - b = ${a - b}"
println "a * b = ${a * b}"
println "a / b = ${a / b}"
```

```
println "\nType conversion:"
```

```
def text = "42"
def number = text as Integer

println "Before: $text -> ${text.getClass()}"
println "After:  $number -> ${number.getClass()}"
println "number + 8 = ${number + 8}"
```

Фрагмент вывода

```
a / b = 3.3333333333
number + 8 = 50
```

Функции и методы

Без return

- Неявный возврат
- Последнее выражение метода становится результатом
- Аргументы передают данные внутрь метода

```
def add(a, b) {  
    a + b  
}  
  
def multiply(a, b) {  
    a * b  
}
```

```
def greet(String name) {  
    "Hello, $name!"  
}  
  
println add(5, 3)  
println multiply(5, 3)  
println greet("Alice")
```

Фрагмент вывода

```
8      15      Hello, Alice!
```

Замыкания Closures

Closure

- Логика как значение
- Блок кода – объект: его можно сохранить и вызвать
- each принимает closure и вызывает для элементов

```
def square = { x ->
  x * x
}

println "Square of 5: ${square(5)}"
```

```
def numbers = [1, 2, 3, 4, 5]

println "\nElements:"

numbers.each { number ->
  println number
}
```

Вывод по блокам

```
Square of 5: 25
Elements: 1 2 3 4 5
```

Условия и Groovy Truth

Groovy Truth

- Значение в условии

- Пустой список – false; непустой список – true
- if проверяет значение без сравнения с true

```
def age = 20

if (age >= 18) {
    println "Adult"
} else {
    println "Minor"
}

def names = []
```

```
if (names) {
    println "The list is not empty"
} else {
    println "The list is empty"
}

names << "Alice"

if (names) {
    println "Now the list is not empty: $names"
}
```

Фрагмент вывода

```
Adult
The list is empty
Now the list is not empty: [Alice]
```

Циклы и итераторы

times / each

- Итерации без счётчика

- times повторяет действие; each обходит элементы
- for обходит диапазон; it – параметр closure

```
println "for:"  
  
for (i in 1..5) {  
    println i  
}  
  
println "\ntimes:"
```

```
5.times {  
    println it  
}  
  
println "\neach:"  
  
[10, 20, 30].each {  
    println it  
}
```

Вывод по блокам

```
for: 1 2 3 4 5  
times: 0 1 2 3 4  
each: 10 20 30
```

Коллекции

findAll → collect

• [1–5] → [2, 4] → [4, 16]

- findAll оставляет элементы, подходящие под условие
- collect преобразует каждый отобранный элемент

```
def numbers = [1, 2, 3, 4, 5]

def user = [
  name: "Alice",
  age: 20
]

println "Numbers: $numbers"
```

```
println "User: $user"
println "Name: ${user.name}"

def result = numbers
  .findAll { it % 2 == 0 }
  .collect { it * it }

println "Squares of even numbers: $result"
```

Фрагмент вывода

```
Name: Alice
Squares of even numbers: [4, 16]
```

Итоговый пример

Анализ списка

- Метод + closure

- average вычисляет средний балл: $28 / 7 = 4$
- findAll отбирает оценки ≥ 4 : пять из семи

```
def grades = [5, 4, 3, 5, 2, 4, 5]

def average(List<Integer> values) {
    values.sum() / values.size()
}
```

```
def goodGrades = grades.findAll { it >= 4 }

println "All grades: $grades"
println "Average grade: ${average(grades)}"
println "Good grades: $goodGrades"
println "Number of good grades: ${goodGrades.size()}"
```

Фрагмент вывода

```
Average grade: 4
Good grades: [5, 4, 5, 4, 5]
Number of good grades: 5
```

Итоги первой части

•Платформа JVM

•Логика программы

•Обработка данных

- Groovy выполняется на JVM и использует библиотеки Java
- Методы и closures позволяют выделять и передавать логику
- Groovy Truth упрощает условия, методы коллекций — обработку данных
- Изученные конструкции объединены в программе анализа оценок

•Следующая часть:
классы

•Пакеты и организация
кода

•Обработка исключений

Список литературы

- Apache Groovy — официальная документация
<https://docs.groovy-lang.org/docs/latest/html/documentation/>
- Apache Groovy — материалы о выпусках языка
<https://groovy-lang.org/releases.html>
- Apache Incubator — статус проекта Groovy
<https://incubator.apache.org/projects/groovy.html>
- Docker Hub — официальный образ Groovy
https://hub.docker.com/_/groovy
- Oracle — обновления языка Java SE 25
<https://docs.oracle.com/en/java/javase/25/language/>